

Actividad 1.1.3:Análisis Comparativo de Algoritmos de Ordenamiento

Luis Fernando Salazar Hernández – A01738527

Instituto Tecnológico y de Estudios Superiores de Monterrey

Diseño de sistemas embebidos avanzados

César Torres Huitzil

12 de octubre de 2026

1. Descripción de la actividad..... 3
2. Metodología y Decisiones de Implementación.....3
3. Explicación y Seudocódigo de los Algoritmos..... 4
3.1. Insertion Sort.....4
3.2. Bubble Sort..... 4
3.3. Merge Sort..... 5
4. Evidencia de Ejecución y Cuadro Comparativo..... 6
5. Conclusiones y Reflexión.....7
6. Anexos y Código Fuente..... 8

1. Descripción de la actividad

En esta actividad revisé cómo funcionan e implementé tres algoritmos para ordenar arreglos en Python: Insertion Sort, Bubble Sort y Merge Sort (que usa la idea de divide y vencerás). El objetivo principal fue medir su eficiencia contando de forma precisa cuántas comparaciones hacen entre los elementos.

Para ver cómo se comportan según el tamaño de la lista, probé con arreglos de 10, 100, 1,000 y 10,000 elementos. Además, en cada tamaño probé tres casos diferentes de entrada:

- Un arreglo desordenado (con datos al azar).
- Un arreglo ordenado de forma ascendente.
- Un arreglo ordenado de forma descendente.

Por último, comparé los resultados obtenidos en una tabla comparativa creada a partir de las pruebas que ejecuté en Google Colab (al final del documento incluyo el enlace al código).

2. Metodología y Decisiones de Implementación

Durante el desarrollo hice varios cambios en el código, pero uno de los más importantes fue la forma en que generé y compartí los arreglos para las pruebas.

Al principio, cada algoritmo generaba sus propios datos aleatorios por su cuenta. Sin embargo, para que las pruebas fueran completamente justas y parejas, pasé la creación de los arreglos a una parte global al inicio del programa. De este modo, si creo un arreglo de N = 10,000 elementos, se genera una sola vez y los tres métodos trabajan sobre ese mismo conjunto.

Para no alterar los datos originales durante la ejecución, usé la función .copy(). Esto es crucial porque en Python las variables funcionan por referencia en memoria. Si un algoritmo ordena la lista original directamente, el siguiente recibiría una lista que ya está ordenada y los resultados del conteo de comparaciones se arruinarían. Sacarle una copia exacta a la lista con .copy() me asegura que cada algoritmo empiece exactamente con los mismos datos desordenados.

Además, puse los incrementos de los contadores (comparaciones += 1) justo antes de evaluar las condiciones donde se comparan los datos del arreglo, dejando fuera del conteo los bucles de control.

3. Explicación y Seudocódigo de los Algoritmos

En esta sección se detalla cada uno de los algoritmos utilizados, describiendo su funcionamiento y su lógica principal.

3.1. Insertion Sort

Este método construye la lista ordenada elemento por elemento. Presenta un rendimiento óptimo con arreglos pequeños o con listas previamente ordenadas (alcanzando una complejidad de O(n)). Sin embargo, cuando los datos se encuentran desordenados, su rendimiento disminuye a $O(n^2)$. Su principal ventaja es que no requiere memoria adicional, por lo que su complejidad espacial es O(1).

Algorithm 2 INSERTION-SORT(A, n)

1: for i = 2 to n do
2: key = A[i]
3: j = i - 1
4: while j > 0 and A[j] > key do
5: A[j + 1] = A[j]
6: j = j - 1
7: end while
7: A[j + 1] = key
9: end for

3.2. Bubble Sort

Es un algoritmo muy conocido pero bastante ineficiente. Lo que hace es comparar e intercambiar repetidamente los elementos contiguos que estén desordenados. Como utiliza dos ciclos anidados, su complejidad promedio y en el peor escenario es de $O(n^2)$. Le agregué una bandera de optimización (intercambiado = False) para detener la ejecución si detecta que el arreglo ya quedó ordenado, logrando que su mejor caso sea de O(n).

Algorithm 3 BUBBLE-SORT(A, n)

1: for i = 1 to n - 1 do
2: for j = 1 to n - i do
3: if A[j] > A[j + 1] then
4: exchange A[j] with A[j + 1]

5: end if

6: end for

7: end for

3.3. Merge Sort

Es un algoritmo bastante rápido que usa la estrategia de "divide y vencerás". Parte la lista a la mitad recursivamente hasta quedarse con arreglos de un solo elemento, y luego los va uniendo de forma ordenada. Esto le da una excelente complejidad temporal constante de O(n log n), aunque requiere memoria adicional de O(n).

Algorithm 4 MERGE-SORT(A, p, r)

1: if p < r then

2: q ← ⌊(p + r)/2⌋

3: MERGE-SORT(A, p, q)

4: MERGE-SORT(A, q + 1, r)

5: MERGE(A, p, q, r)

6: end if

Algorithm 5 MERGE(A, p, q, r)

1: n1 = q - p + 1

2: n2 = r - q

3: let L[1 ... n1] and R[1 ... n2] be new arrays

4: for i = 1 to n1 do

5: L[i] = A[p + i - 1]

6: end for

7: for j = 1 to n2 do

8: R[j] = A[q + j]

9: end for

10: i = 1

11: j = 1

12: for k = p to r do

13: if i <= n1 and (j > n2 or L[i] <= R[j]) then

14: A[k] = L[i]

15: i = i + 1

16: else

17: A[k] = R[j]

18: j = j + 1

19: end if

20: end for

4. Evidencia de Ejecución y Cuadro Comparativo

Después de correr las pruebas, guardé la cantidad exacta de comparaciones. En los resultados se nota claramente cómo las operaciones crecen de forma cuadrática en los métodos tradicionales, mientras que Merge Sort escala de forma mucho más amigable gracias a su comportamiento logarítmico.

Tamaño (n)	Tipo de Arreglo	Insertion Sort (Comparaciones)	Bubble Sort (Comparaciones)	Merge Sort (Comparaciones)
10	Aleatorio	34	44	22
10	Ascendente	9	9	19
10	Descendente	45	45	15
100	Aleatorio	2,335	4,929	538
100	Ascendente	99	99	356
100	Descendente	4,950	4,950	316
1,000	Aleatorio	252,037	496,650	8,711
1,000	Ascendente	999	999	5,044
1,000	Descendente	499,500	499,500	4,932
10,000	Aleatorio	25,104,462	49,982,120	120,464
10,000	Ascendente	9,999	9,999	69,008
10,000	Descendente	49,995,000	49,995,000	64,608

5. Conclusiones y Reflexión

Revisar los datos de la tabla nos permite entender muy fácilmente las diferencias reales de rendimiento entre cada tipo de complejidad.

A partir de estos valores se evidencia por qué Insertion Sort y Bubble Sort no son recomendables para procesar grandes volúmenes de datos. Por ejemplo, al evaluar 10,000 elementos en el peor caso posible (un arreglo ordenado de forma descendente), Bubble Sort realizó cerca de 50 millones de comparaciones. En contraste, Merge Sort resolvió el mismo caso con poco más de 65,000 operaciones, lo que demuestra la eficacia de la estrategia de divide y vencerás para optimizar el cómputo.

Un aspecto relevante observado en las pruebas fue el comportamiento de Insertion Sort y Bubble Sort cuando la lista de entrada se encontraba ordenada. Gracias a la optimización de interrupción temprana al no detectar intercambios, ambos métodos concluyeron el proceso de forma eficiente ejecutando únicamente O(n) comparaciones (9, 99, 999 y 9,999 respectivamente). En este escenario específico, superaron en desempeño a Merge Sort, debido a que la estructura recursiva de este último no permite detener la ejecución de manera anticipada.

Si llevamos esto al terreno de los sistemas embebidos, vemos lo importante que es cuidar los recursos del hardware. Aunque Merge Sort es mucho más rápido y ahorra energía al procesar, necesita memoria extra para copiar los subarreglos (O(n)) y recarga la pila por sus llamadas recursivas. En microcontroladores con poca memoria RAM, esto podría causar un desbordamiento de pila (stack overflow).

Por otro lado, si un sensor requiere procesar un volumen reducido de datos en un solo bloque (como calcular la mediana de 20 lecturas de temperatura), Insertion Sort representa una alternativa adecuada. Opera directamente sobre el arreglo original sin requerir asignación de memoria adicional (complejidad espacial O(1)) y sin incurrir en la sobrecarga de la recursión, lo que evidencia que la selección del algoritmo idóneo depende del contexto y las restricciones del sistema.

6. Anexos y Código Fuente

El código completo desarrollado para esta práctica, incluyendo la generación de los arreglos globales, la implementación de los algoritmos con contadores y la lógica de pruebas iterativas, se encuentra disponible para su revisión y ejecución en el siguiente cuaderno de Google Colab:

activity 113_comparative analysis of sorting algorithms.ipynb